

Flyway & Liquibase DB Migrations

Sub-Chapter 04.09 | Chapter 4: JPA & Hibernate

Beginner-Friendly Guide to Database Schema Versioning

Why DB Migration Tools?	Versioning schema changes like code
Flyway Versioned Scripts	V1___, V2___ naming convention
flyway_schema_history Table	Tracks applied migrations & checksums
Liquibase Changelog	XML/YAML changesets with author & id
Spring Boot Integration	Auto-runs migrations on startup
Flyway vs Liquibase	Feature comparison table
Rollback Strategies	Partial Flyway undo vs Liquibase rollback

1 Why DB Migration Tools?

Schema changes need versioning like code

Every application evolves over time. New features need new columns, renamed tables, or dropped indexes. Without a migration tool, developers manually run ALTER TABLE statements on every environment — dev, test, staging, and production — leading to drift, forgotten changes, and hard-to-reproduce bugs. DB migration tools solve this by versioning every schema change in files that are committed to source control and applied automatically and consistently across all environments.

Example: Without a migration tool (risky manual approach)

```
-- Developer A runs this locally:
ALTER TABLE users ADD COLUMN email VARCHAR(255);

-- Developer B forgets to run it.
-- Production may never get it.
-- Bug: "Column email not found" on staging.
```

Example: With a migration tool (safe, versioned approach)

```
-- File: V2__add_email_to_users.sql
-- Committed to Git, applied automatically on every environment.
ALTER TABLE users ADD COLUMN email VARCHAR(255) NOT NULL DEFAULT '';
```

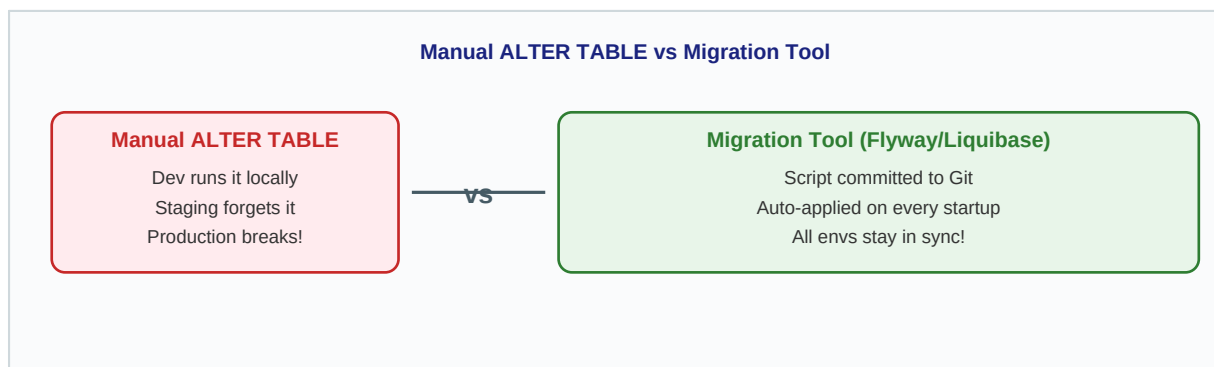


Fig 1.1: Manual approach vs migration tool approach

Key Rules

- Schema changes must be versioned in source control alongside code.
- Migration tools ensure every environment (dev/test/prod) runs the same scripts.
- Never manually run ALTER TABLE in production without a versioned script.
- Migration files are immutable once applied — never edit a deployed migration.
- Each script runs exactly once per environment (tracked by the tool).

Q1. Why not just manually ALTER TABLE on each database?

Manual changes cause environment drift. If one environment misses a change, bugs appear only in that environment. Migration tools automate and track every change.

Q2. What does 'schema versioning' mean?

It means each database change is stored in a numbered file (like V1, V2) committed to Git, so the history of schema changes is visible and reproducible.

Q3. Can migration tools handle both DDL and DML?

Yes. They can run CREATE TABLE (DDL) and INSERT/UPDATE (DML) statements. DDL changes are most common, but seeding initial data is also supported.

Q4. What happens if two developers add conflicting migrations?

The tool detects the version conflict on startup and refuses to run. Developers must resolve the conflict by renumbering or merging the scripts.

Q5. Do migration tools work with all databases?

Flyway and Liquibase both support major databases: MySQL, PostgreSQL, Oracle, SQL Server, H2, and more. Some SQL syntax may differ per database.

2 Flyway Versioned SQL Scripts

V1__, V2__ naming convention

Flyway applies SQL migration scripts placed in a specific folder (usually `src/main/resources/db/migration`). Scripts must follow a strict naming convention: a version prefix, double underscore, and a description. Flyway reads all unexecuted scripts in version order and runs them sequentially. Once a script is applied, Flyway records it and never runs it again.

Flyway Naming Convention

V{version}__{description}.sql

Examples:

```
V1__create_users_table.sql
V2__add_email_column.sql
V3__create_orders_table.sql
R__seed_reference_data.sql  (repeatable migration)
```

V1__create_users_table.sql

```
CREATE TABLE users (
  id   BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

V2__add_email_column.sql

```
ALTER TABLE users
  ADD COLUMN email VARCHAR(255) NOT NULL DEFAULT '';
```

V3__create_orders_table.sql

```
CREATE TABLE orders (
  id       BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id  BIGINT NOT NULL,
  total    DECIMAL(10,2),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

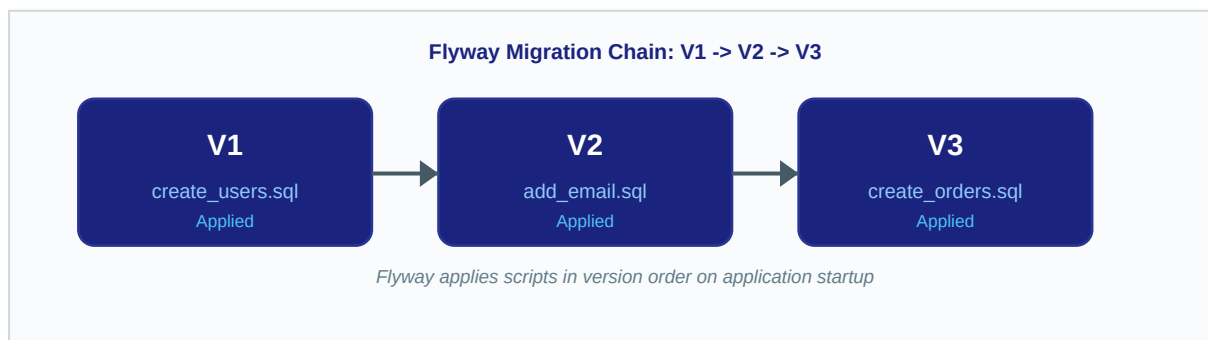


Fig 2.1: Flyway applies V1, V2, V3 in sequence — each exactly once

Key Rules

- Prefix: V (versioned), R (repeatable), U (undo — paid feature).
- Double underscore (__) separates version number from description.
- Version can be 1, 1.1, 2, 20240601 — any sortable numeric format.
- Description uses underscores for spaces: V2__add_email_column.sql.
- Scripts must live in `src/main/resources/db/migration` by default.

- Never rename or edit a script after it has been applied.

Q1. What does the double underscore (__) do in Flyway filenames?

It separates the version prefix (e.g., V2) from the human-readable description. Flyway uses this separator to parse the version number.

Q2. Can I use decimal version numbers like V1.2?

Yes. Flyway supports versions like V1, V1.1, V1.2, V2. They are sorted numerically, not alphabetically, so V1.9 comes before V1.10.

Q3. What are repeatable migrations (R__ prefix)?

Repeatable migrations re-run whenever their content changes. They are useful for views, stored procedures, or seed data that needs refreshing.

Q4. Where should Flyway migration scripts be placed in a Spring Boot project?

By default: src/main/resources/db/migration. This can be changed via the spring.flyway.locations property.

Q5. What happens if I delete a migration script that was already applied?

Flyway will throw an error on startup because the checksum in its history table no longer matches the file. Use spring.flyway.ignore-missing-migrations=true to override.

3 flyway_schema_history Table

Tracks applied migrations, version, checksum

Flyway tracks every applied migration in a special table called `flyway_schema_history` (or `schema_version` in older Flyway versions). This table is created automatically on the first run. It records the version number, script description, type (SQL or Java), checksum (CRC32 hash of the file), the user who ran it, execution time, and whether it succeeded. On every startup Flyway compares applied history against available scripts to decide what still needs to run.

flyway_schema_history table structure (SQL)

```
SELECT installed_rank, version, description,
       type, script, checksum, success
FROM   flyway_schema_history
ORDER BY installed_rank;
-- Sample output:
-- rank | version | description      | checksum   | success
-- 1    | 1       | create users table | 1234567890 | true
-- 2    | 2       | add email column  | 0987654321 | true
-- 3    | 3       | create orders     | 1122334455 | true
```

flyway_schema_history Table

version	description	checksum	success
1	create users table	1234567890	true
2	add email column	0987654321	true
3	create orders	1122334455	true

Flyway auto-creates and updates this table — never edit it manually

Fig 3.1: flyway_schema_history table with version, description, checksum, success columns

Key Rules

- `flyway_schema_history` is created automatically on the first Flyway run.
- The checksum column holds a CRC32 hash of the script file content.
- If a file changes after being applied, Flyway detects the checksum mismatch and fails.
- The success column shows whether the migration completed without error.
- Never manually edit or delete rows from `flyway_schema_history`.
- You can repair broken state using the Flyway 'repair' command.

Q1. What is the checksum column used for?

It stores a CRC32 hash of the migration file. On each startup Flyway re-hashes the file and compares it to detect accidental edits to applied scripts.

Q2. What happens if a migration fails midway?

For databases that support transactional DDL (PostgreSQL), Flyway rolls back the whole script. For others (MySQL), partial changes may remain and must be cleaned up manually.

Q3. How do I fix a failed migration entry in flyway_schema_history?

Use the flyway repair command. It removes failed entries and recalculates checksums, allowing you to fix the script and re-apply it.

Q4. Can I query flyway_schema_history directly?

Yes, it is a regular database table. You can SELECT from it to audit which migrations were applied, when, and by whom.

Q5. What is installed_rank?

It is the sequential order in which migrations were applied, regardless of version. It helps distinguish the physical order of execution.

4 Liquibase Changelog

XML/YAML changeset with author and id

Liquibase uses a changelog file to describe all database changes. Instead of plain SQL files, each change is wrapped in a changeset element with a unique id and author attribute. Changesets can be written in XML, YAML, JSON, or SQL format. Liquibase also supports database-agnostic change types (like `createTable`, `addColumn`) that it translates to the correct SQL for each database.

Liquibase changelog in XML format

```
<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
  xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
    http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-4.0.xsd">

  <changeSet id="1" author="alice">
    <createTable tableName="users">
      <column name="id" type="BIGINT" autoIncrement="true">
        <constraints primaryKey="true"/>
      </column>
      <column name="name" type="VARCHAR(100)"/>
    </createTable>
  </changeSet>

  <changeSet id="2" author="bob">
    <addColumn tableName="users">
      <column name="email" type="VARCHAR(255)"/>
    </addColumn>
  </changeSet>

</databaseChangeLog>
```

Liquibase changelog in YAML format

```
databaseChangeLog:
- changeSet:
  id: 1
  author: alice
  changes:
    - createTable:
      tableName: users
      columns:
        - column:
            name: id
            type: BIGINT
            autoIncrement: true
            constraints:
              primaryKey: true
        - column:
            name: name
            type: VARCHAR(100)
- changeSet:
  id: 2
  author: bob
  changes:
    - addColumn:
      tableName: users
      columns:
```



```
- column:
  name: email
  type: VARCHAR(255)
```

Key Rules

- Every changeset must have a unique combination of id + author.
- Liquibase tracks applied changesets in the DATABASECHANGELOG table.
- Use XML or YAML for database-agnostic changesets; SQL for database-specific ones.
- Never change the id or author of an applied changeset — it breaks the checksum.
- Include multiple changeset files using in a master changelog.

Q1. What is the difference between a Flyway script and a Liquibase changeset?

A Flyway script is a plain SQL file with a versioned filename. A Liquibase changeset is a structured element (XML/YAML) with an id and author, offering database-agnostic change types.

Q2. Can Liquibase use plain SQL instead of XML?

Yes. Liquibase supports SQL changelogs where you write raw SQL inside changeset blocks. This is useful when migrating from Flyway or for complex SQL.

Q3. What table does Liquibase use to track applied changes?

DATABASECHANGELOG. It stores the id, author, filename, dateExecuted, orderExecuted, execType, and MD5SUM of each applied changeset.

Q4. What happens if the same changeset id is used twice?

Liquibase will throw an error because the combination of id + author must be globally unique across all changelog files.

Q5. Can I include separate changelog files in Liquibase?

Yes. Use the includeAll or include tag in a master changelog file to split migrations across multiple files for better organization.

5 Spring Boot Integration

Auto-runs migrations on startup, dependency config

Spring Boot integrates with both Flyway and Liquibase out of the box. Simply add the dependency to your pom.xml or build.gradle and provide the migration files. Spring Boot auto-configures Flyway or Liquibase using your existing DataSource bean. Migrations run automatically before the application context finishes starting, ensuring the database is always up to date when the application serves its first request.

Maven dependency for Flyway (pom.xml)

```
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
```

Maven dependency for Liquibase (pom.xml)

```
<dependency>
  <groupId>org.liquibase</groupId>
  <artifactId>liquibase-core</artifactId>
</dependency>
```

application.properties for Flyway

```
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
spring.flyway.baseline-on-migrate=true
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
```

application.properties for Liquibase

```
spring.liquibase.enabled=true
spring.liquibase.change-log=classpath:db/changelog/db.changelog-master.xml
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=postgres
spring.datasource.password=secret
```

Spring Boot startup flow with Flyway (Java config check)

```
// No extra Java config needed with Spring Boot auto-config.
// Flyway reads your DataSource and migrations automatically.
// To customize programmatically:
@Bean
public FlywayMigrationStrategy migrationStrategy() {
    return flyway -> {
        flyway.repair();
        flyway.migrate();
    };
}
```

Key Rules

- Add flyway-core or liquibase-core dependency — Spring Boot does the rest.
- Migrations run before the application context completes startup.
- spring.flyway.baseline-on-migrate=true handles existing databases without history.
- spring.flyway.locations can point to classpath or filesystem directories.
- Set spring.flyway.enabled=false to disable auto-migration in specific profiles.
- Liquibase change-log path defaults to classpath:db/changelog/db.changelog-master.yaml.

Q1. Does Spring Boot auto-configure Flyway without any Java code?

Yes. Adding flyway-core to the classpath and providing migration scripts is enough. Spring Boot detects the dependency and auto-configures Flyway with your DataSource.

Q2. When exactly do migrations run during Spring Boot startup?

After the DataSource bean is created but before any JPA/Hibernate entity validation. This ensures the schema is ready before the ORM tries to map entities to tables.

Q3. How do I disable Flyway in tests?

Add `spring.flyway.enabled=false` in your test `application.properties` or use `@TestPropertySource` to override the property for specific test classes.

Q4. Can Spring Boot run both Flyway and Liquibase together?

Technically yes, but it is not recommended. Choose one tool per project to avoid conflicts in schema history tracking.

Q5. What is `baseline-on-migrate` used for?

It tells Flyway to mark all existing schema objects as already migrated when you introduce Flyway to an existing database that has no migration history yet.

6 Flyway vs Liquibase

Feature comparison table

Both Flyway and Liquibase are mature, widely used database migration tools with excellent Spring Boot support. The choice between them depends on your team's preferences, the complexity of rollback needs, and whether you prefer plain SQL files or structured changelog formats.

Feature	Flyway	Liquibase
Migration format	Plain SQL files (primarily)	XML, YAML, JSON, or SQL
Naming convention	V{ver}__{desc}.sql required	id + author in changeset
History table	flyway_schema_history	DATABASECHANGELOG
Rollback support	Undo scripts (paid / Teams edition)	Built-in rollback tags (free)
Database-agnostic DDL	No — plain SQL only	Yes — createTable, addColumn etc.
Spring Boot support	Yes — auto-configured	Yes — auto-configured
Learning curve	Very simple — just SQL files	Moderate — XML/YAML syntax
CLI tool	Flyway CLI (free)	Liquibase CLI (free)
Java migrations	Yes — JavaMigration interface	Yes — custom Change class
Repeatable migrations	Yes — R__ prefix	Yes — runOnChange: true
Dry run	Paid feature	updateSQL command (free)
Community / license	Apache 2.0 (core), paid Teams/Enterprise	Apache 2.0 (open source)

Key Rules

- Flyway is simpler: just name your SQL files correctly and drop them in a folder.
- Liquibase is more powerful for rollbacks and database-agnostic migrations.
- Both tools integrate identically with Spring Boot via a single dependency.
- Choose Flyway for simplicity; choose Liquibase for advanced rollback needs.
- Both tools support Java-based migrations for complex logic that cannot be expressed in SQL.

Q1. Which tool is easier for beginners?

Flyway is generally easier. You only need to create correctly named SQL files. Liquibase requires learning its XML/YAML changeset syntax first.

Q2. Does Liquibase support plain SQL?

Yes. Liquibase can use SQL changelogs that contain raw SQL, making migration from Flyway straightforward.

Q3. Is rollback free in Liquibase?

Yes. Liquibase provides rollback tags and rollback commands in the free open-source version. Flyway requires the paid Teams edition for undo scripts.

Q4. Can I switch from Flyway to Liquibase mid-project?

It is possible but complex. You would need to create a Liquibase baseline changeset that represents the current schema state, then add new changesets going forward.

Q5. Which tool do most large enterprises use?

Both are used widely. Liquibase is often chosen in enterprises needing rollbacks and database-agnostic scripts. Flyway is preferred for its simplicity and SQL clarity.

Rolling back a database migration is one of the hardest problems in schema management. Unlike application code, database changes are often destructive (dropping columns loses data). Both Flyway and Liquibase offer rollback mechanisms, but with different approaches and licensing. Understanding your rollback strategy before choosing a tool is important for production-grade applications.

Flyway rollback: Undo scripts (Teams edition)

```
-- Forward migration: V2__add_email_column.sql
ALTER TABLE users ADD COLUMN email VARCHAR(255);

-- Undo migration (Flyway Teams): U2__add_email_column.sql
ALTER TABLE users DROP COLUMN email;

-- Run undo via CLI:
-- flyway undo
-- (Reverts the most recently applied versioned migration)
```

Flyway community rollback: manual compensating migration

```
-- No undo feature in free Flyway.
-- Write a new forward migration to reverse the change:

-- V3__remove_email_column.sql
ALTER TABLE users DROP COLUMN email;

-- This is the recommended community approach:
-- always move forward, never backward.
```

Liquibase rollback: built-in rollback tag (free)

```
<changeSet id="2" author="bob">
  <addColumn tableName="users">
    <column name="email" type="VARCHAR(255)" />
  </addColumn>
  <rollback>
    <dropColumn tableName="users" columnName="email" />
  </rollback>
</changeSet>

-- Run rollback via CLI:
-- liquibase rollbackCount 1
-- (Reverts the last 1 changeset using its rollback block)
```

Liquibase rollback to a tag

```
-- Tag the current state before a release:
-- liquibase tag v1.0

-- After a bad deployment, roll back to the tag:
-- liquibase rollback v1.0

-- Liquibase executes the rollback blocks of all
-- changesets applied after the tag, in reverse order.
```

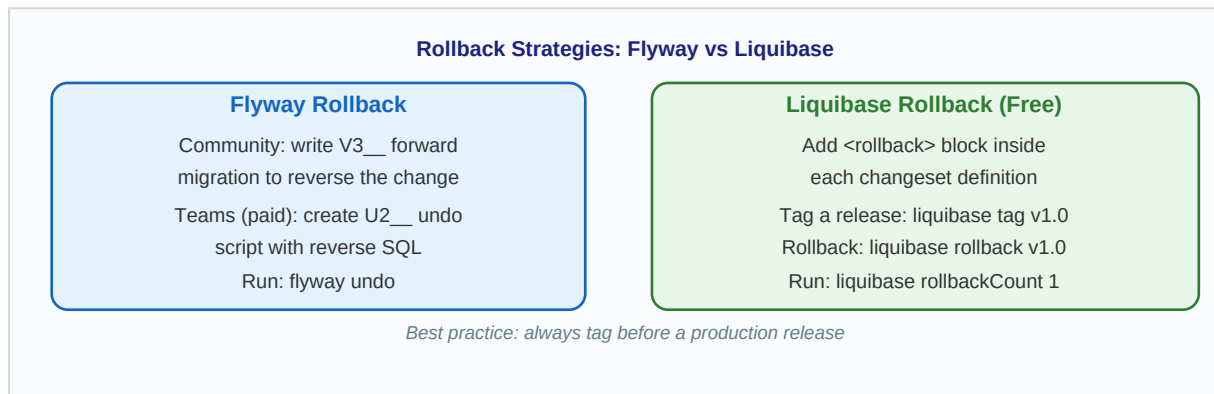


Fig 7.1: Flyway (paid undo) vs Liquibase (free rollback blocks)

Key Rules

- Flyway community edition has no built-in rollback — use compensating forward migrations.
- Flyway Teams edition provides U__ undo scripts that mirror each V__ migration.
- Liquibase provides free rollback using blocks inside changesets.
- Use 'liquibase tag v1.0' before every production release as a safety checkpoint.
- Not all changes are safely reversible — dropping a column permanently loses data.
- The safest strategy is always: expand, then contract (never drop before migrating data).

Q1. Does free Flyway support rollback?

No. The rollback (undo) feature requires Flyway Teams or Enterprise edition. In the community edition, you write a new forward migration that reverses the change.

Q2. What is an expand-and-contract pattern?

It is a safe rollback strategy: first add the new column (expand), migrate data, then drop the old column (contract) in a later release. This avoids destructive rollbacks.

Q3. What does 'liquibase rollbackCount 1' do?

It reverses the most recently applied changeset by executing its block. You can specify any number to roll back multiple changesets.

Q4. Is tagging a mandatory step in Liquibase?

No, but it is strongly recommended before production releases. Tags give you a named rollback point instead of counting changesets manually.

Q5. Can Flyway undo scripts be incomplete?

Yes, and that is a risk. If the undo script does not perfectly reverse the forward migration (e.g., a DROP TABLE loses data), the rollback may leave the database in an inconsistent state.